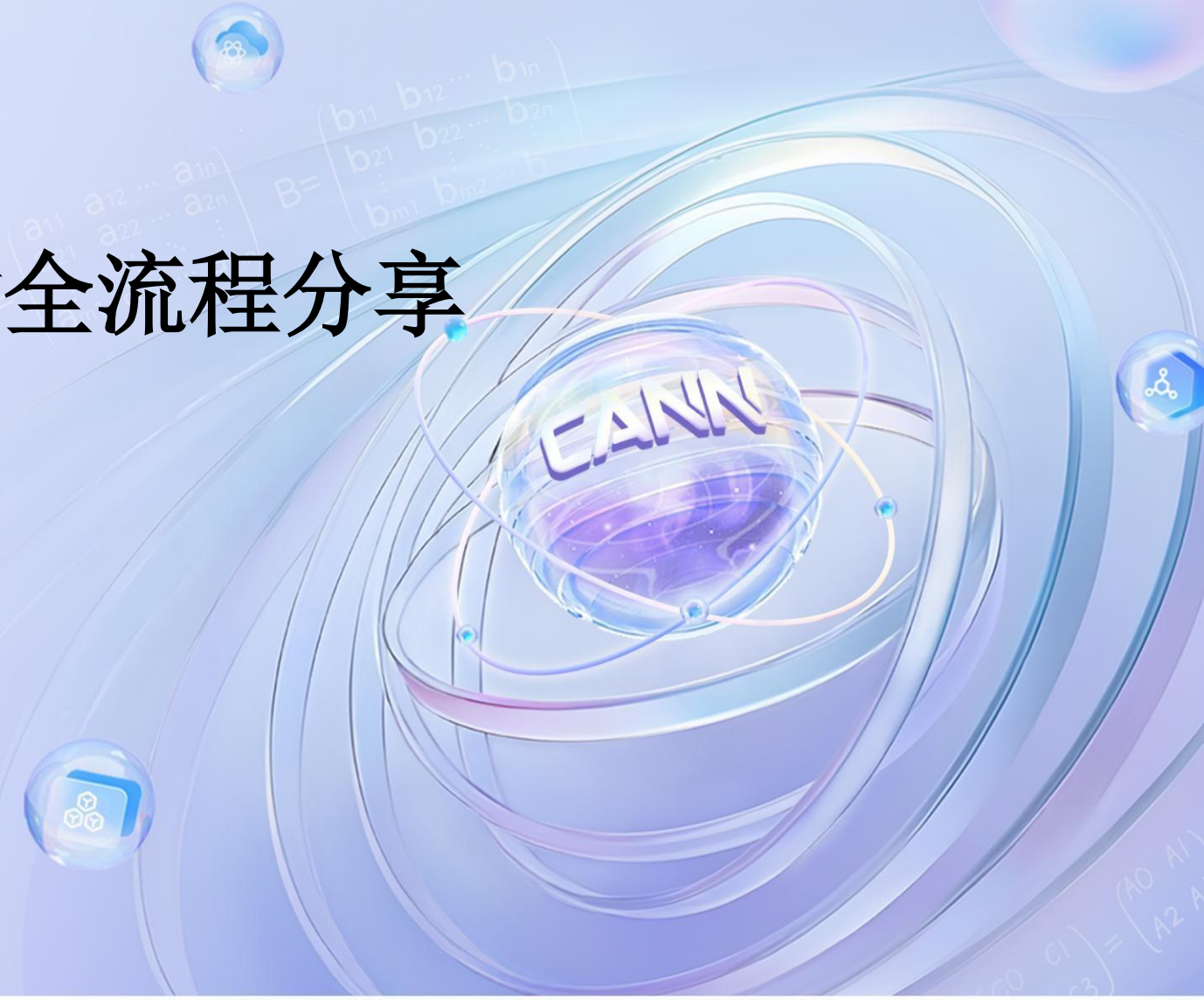
CANNBot算子测试全流程分享



CANN

修订记录

版本号	修订日期	修订人	修订页码	修订内容摘要
V2.0	2026/08/26	梁冰龙	全部	初版内容

目录

Part 1 为什么用Agent进行算子测试

Part 2 CANNBot算子测试全流程设计

Part 3 算子测试全流程实操

Part 4 开放性讨论

为什么用Agent进行算子测试

算子测试面临的挑战

- 算子参数组合爆炸，参数间依赖关系复杂
- 融合算子复杂度高，标杆实现困难
- 算子调用链路深，问题定位困难
- 算子测试流程固定，不同算子需要独立分析

Agent的优势

- LLM自动分析参数的组合和依赖关系
- LLM具备强大的代码能力和先验知识
- LLM强大的推理能力，能够快速定位缺陷根源
- Agent擅长处理流程固定，步骤不确定的任务

结论：用 Agent 将规范化的测试流程自动化、提升质量和效率

CANN

目录

Part 1 为什么用Agent进行测试

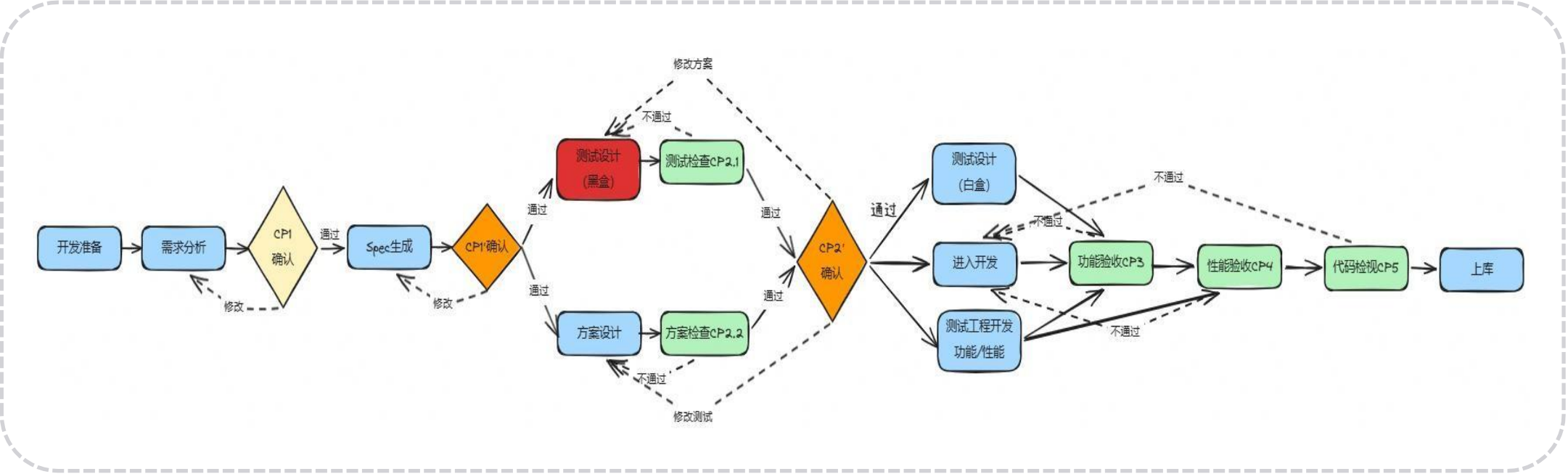
Part 2 CANNBot算子测试全流程设计

Part 3 算子测试全流程实操

Part 4 开放性讨论

引言：CANNBot算子生成中的测试挑战

- 在CANNBot中，旨在**根据**算子**需求Spec自动生成**黑盒测试**方案和用例**；当前依赖人工设计的现状无法支撑该目标达成；
- CANNBot算子测试为cannbot提供智能化验证以及cannbot交付的算子的测试验证；



CANNBot 是面向 CANN 开发的用于提升开发效率的系列智能体，目前已覆盖 Ascend C / PyPTO / TileLang / Triton 算子开发流程和 NPU 模型推理端到端优化

算子测试5大核心流程



全流程：测试设计 -> 测试编码 -> 用例执行 -> 结果分析 -> 测试报告

- CANNBot算子测试全流程：基于算子测试的5大核心流程构建SKILL技能，设置严格的准入准出标准和harness机制。
- CANNBot算子测试新增覆盖度评估模块，多维度评估AI测试的完备度，确保AI测试达到手工测试水准。

https://gitcode.com/cann

CANN

CANNBot 算子测试全流程架构设计

T **TEAMS（应用编排层）**
应用编排

- **Operator-test** 算子测试
- Cann-analysis 问题定位

A **AGENTS（角色执行层）**
角色执行

- test-design 测试设计
- test-coding 测试编码
- test-execute 测试执行
- op-result-analysis 问题分析

S **SKILLS（知识能力层）**
知识能力

- ascendc-st-design 测试设计
- golden-generator 标杆生成
- ttk-execute ttk框架执行
- aclnn-execute aclnn框架执行
- report-generator 报告生成
- result-analysis 结果分析
- coverage-evaluation 覆盖度评估

编排层： Team编排Agents，驱动7阶段标准化开发流程

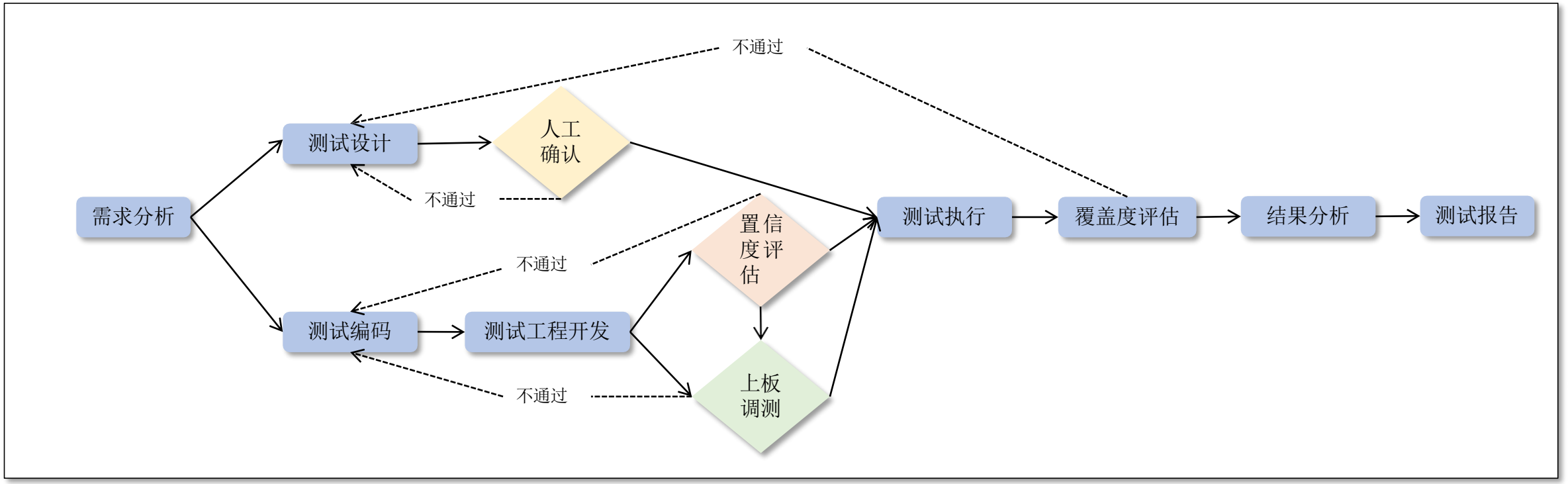
执行层： Agents绑定Skills，执行测试设计、标杆生成、用例执行、结果分析等任务

能力层： Skills提供7+标准化能力，支撑端到端自动化

https://gitcode.com/cann

CANN

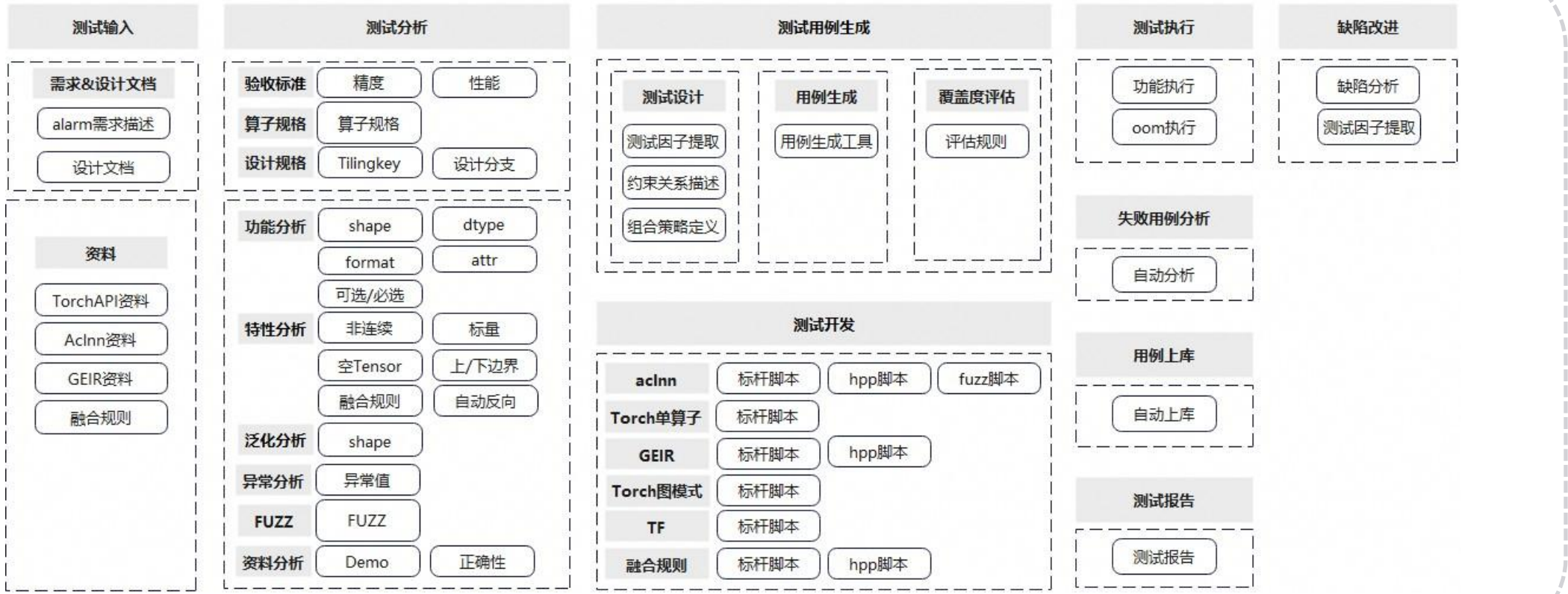
CANNBot 算子测试全流程



- 算子测试全流程：需求分析 -> 测试设计 -> 测试编码 -> 测试执行 -> 结果分析 -> 覆盖度评估 -> 测试报告
- 测试设计：根据需求规格生成测试用例，自主校验和多轮迭代修复，人工评审确认正确性和完备性
- 测试编码：基于上板调测和置信度评分评估测试编码的可用性
- 覆盖度评估：基于测试结果的覆盖度评估，评估测试设计的完备性
- 各流程定义关键的交付件用于流程之间上下文传递和关键信息检查

CANNBot 算子测试：算子测试全景图

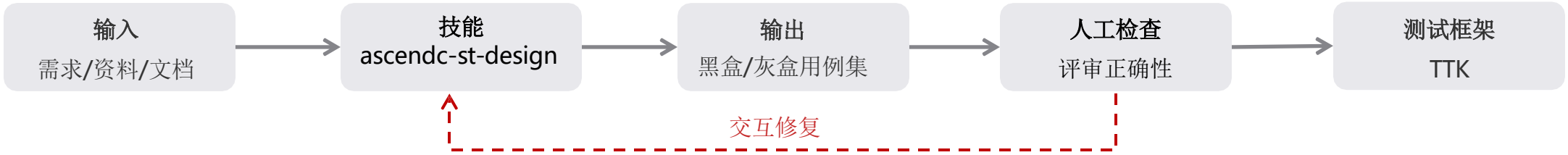
- 算子测试需覆盖用户场景（模型规格），功能，特性，异常，fuzz，泛化，资料等多个维度，每个维度都要进行精度和性能测试；
- 算子测试需要对输入的tensor做覆盖验证，包括shape，dtype，format等，对接口入参进行参数组合的覆盖测试；



https://gitcode.com/cann

CANNBot算子测试：测试设计

根据输入资料使用测试设计skill生成多维度算子测试用例；人工评审确认正确性和完备度（稳定后消除）；



测试设计技能清单

测试类型	技能名称	功能描述	覆盖维度	输出交付件
功能测试	ascendc-st-design	读取Profiling数据，映射转换生成算子参数，覆盖真实业务场景	用户场景	网络用例
	ascendc-st-design	读取算子资料，对参数进行pairwise组合并泛化shape全空间，覆盖全量参数组合边界	功能、特性、泛化、资料	功能泛化用例
DFX测试	ascendc-st-design	读取用户资料，对算子参数使用secodeFuzz算法进行变异探索，探索异常输入的稳定性	FUZZ	FUZZ用例
	ascendc-st-design	读取用户资料，设计异常用例，验证拦截和错误码等	异常	异常用例
灰盒测试	ascendc-st-design	读取设计文档与代码实现，等价类划分覆盖Tiling模板逻辑，覆盖内部处理分支	灰盒	灰盒用例

CANNBot算子测试：功能泛化skill详解

根据需求Spec或用户资料，自动生成算子用例，覆盖功能、特性、泛化、资料维度

人工编写测试用例

1. **人工阅读需求文档**
   阅读上下文，逐条理解算子参数定义，并补齐隐含语义
2. **测试因子定义**
   手动梳理所有输入参数及其取值范围，并在yaml定义因子
3. **约束定义**
   手动梳理参数间的约束关系，并在yaml中描述，工作量大
4. **用例生成**
   功能/泛化：根据测试因子及因子约束进行pairwise组合
   特性/资料：人工分析并生成用例

范式转变

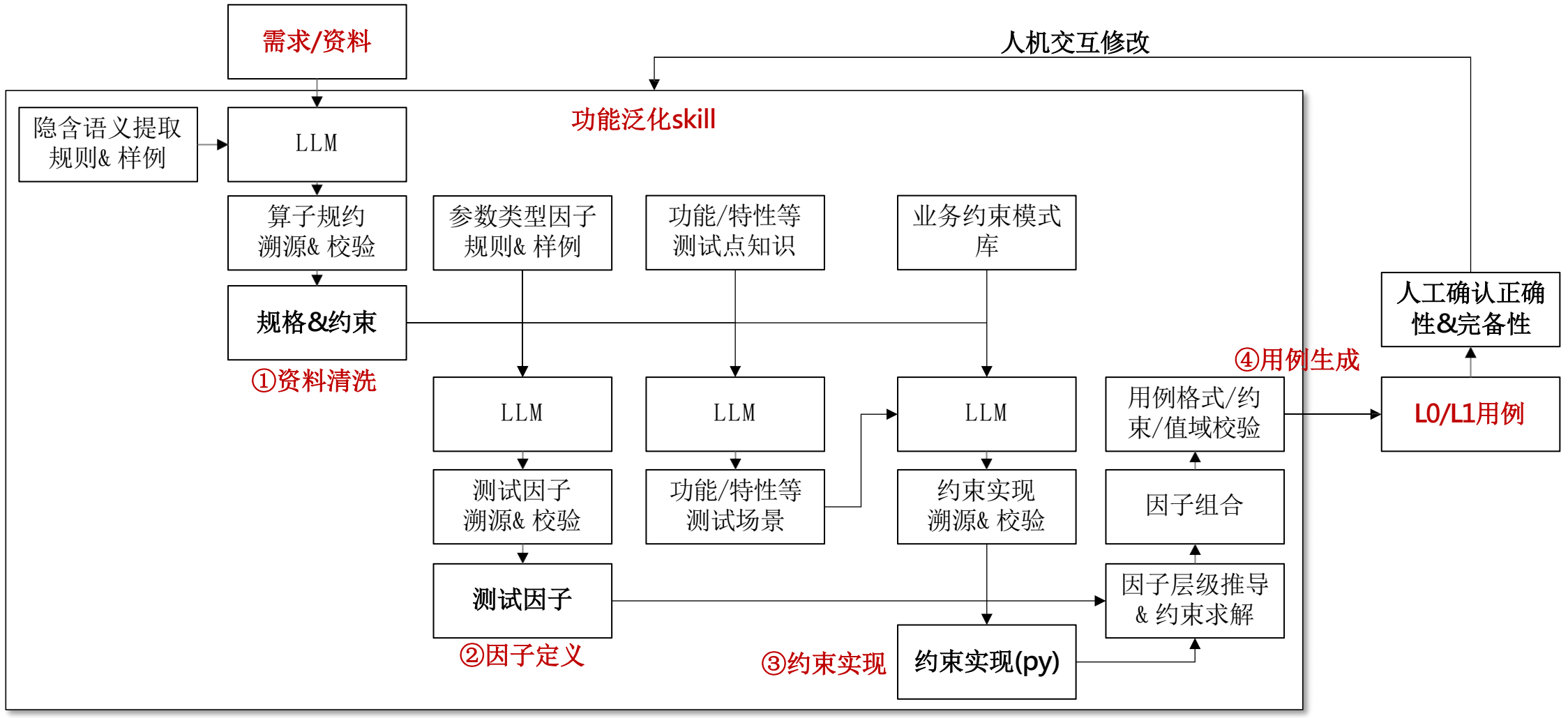
功能泛化 Skill

1. **资料清洗**
   LLM自动汇总参数说明，结合计算公式自动补齐隐含语义
   输出：算子参数规约；资料语义冲突中断反馈人工确认
2. **测试因子定义**
   LLM基于参数类型及因子规则，根据规约自动定义测试因子
   输出：测试因子及因子值域
3. **约束定义**
   LLM分析功能/特性/泛化子场景，通过约束表达/实现
   输出：测试因子取值约束
4. **用例生成**
   固化py自动求解因子值，并采用pairwise组合生成用例
   输出：L0/L1用例

范式转变：从“人工设计用例”转变为“基于LLM理解约束，自动分析并推导用例”

CANN

CANNBot算子测试：功能泛化skill详解



- **资料清洗**：补齐算子隐含语义，输出参数规约
- **因子定义**：基于参数类型因子规则，定义因子及值域
- **约束实现**：分析功能/特性等测试场景，用约束表达；并用py函数实现约束
- **用例生成**：推导因子层级并求解约束，并基于pairwise组合生成用例

CANNBot算子测试：测试编码

Step1： 加载配置文件，获取算子名和测试框架

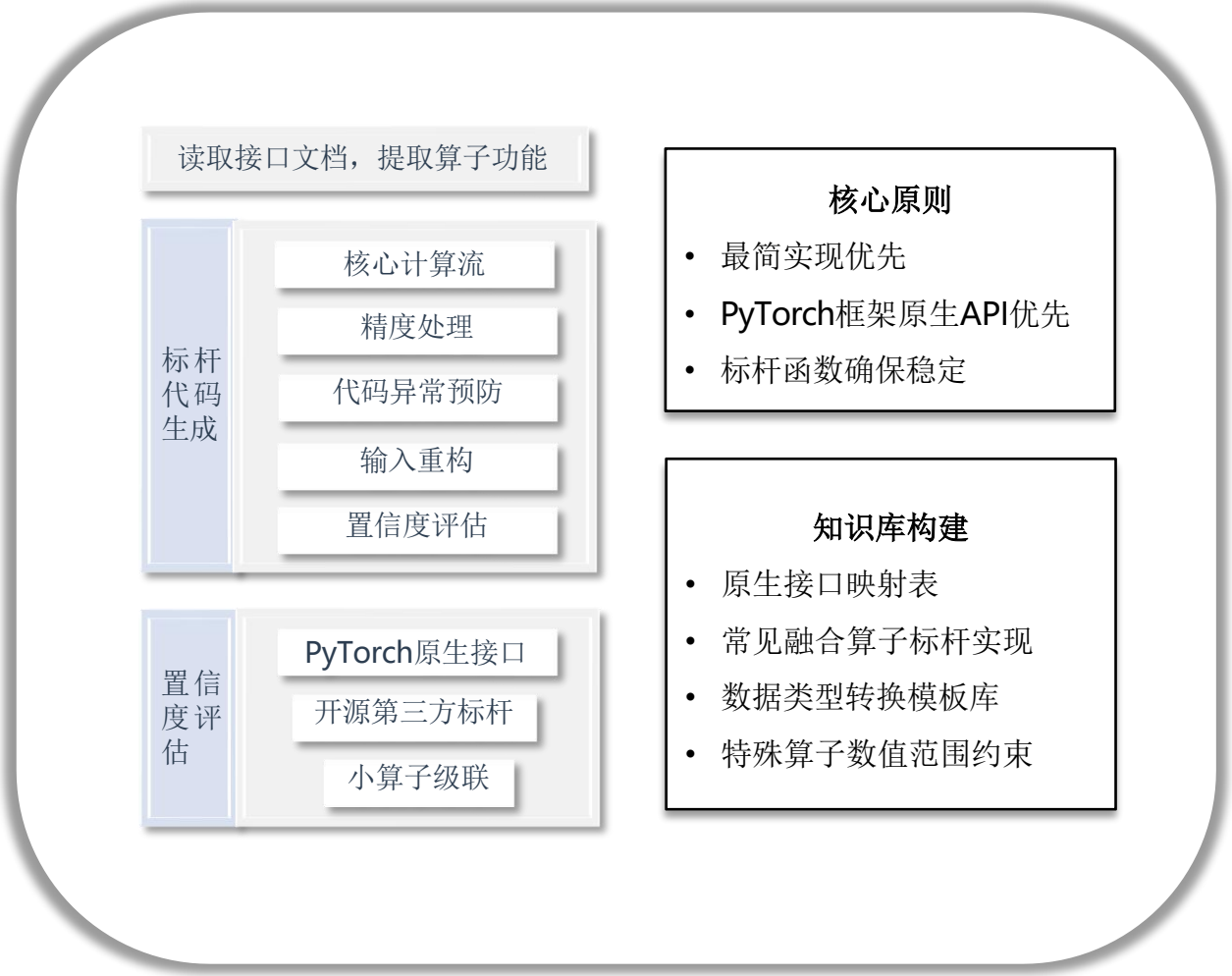
Step2： 加载算子接口文档，确认精度标准

Step3： 标杆代码生成（正确性比对）

Step4： 输入数据重构（可选）

Step5： 测试工程开发

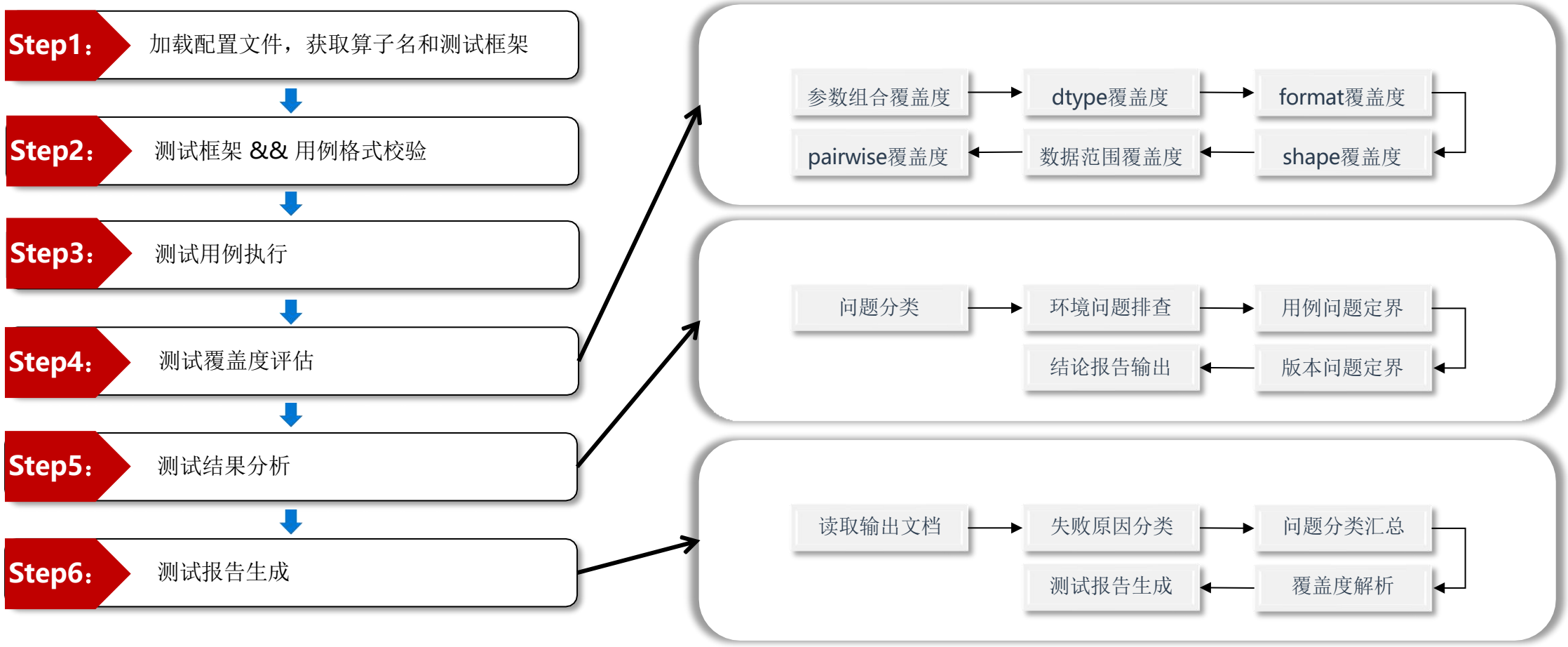
Step6： 上板调测



https://gitcode.com/cann

CANN

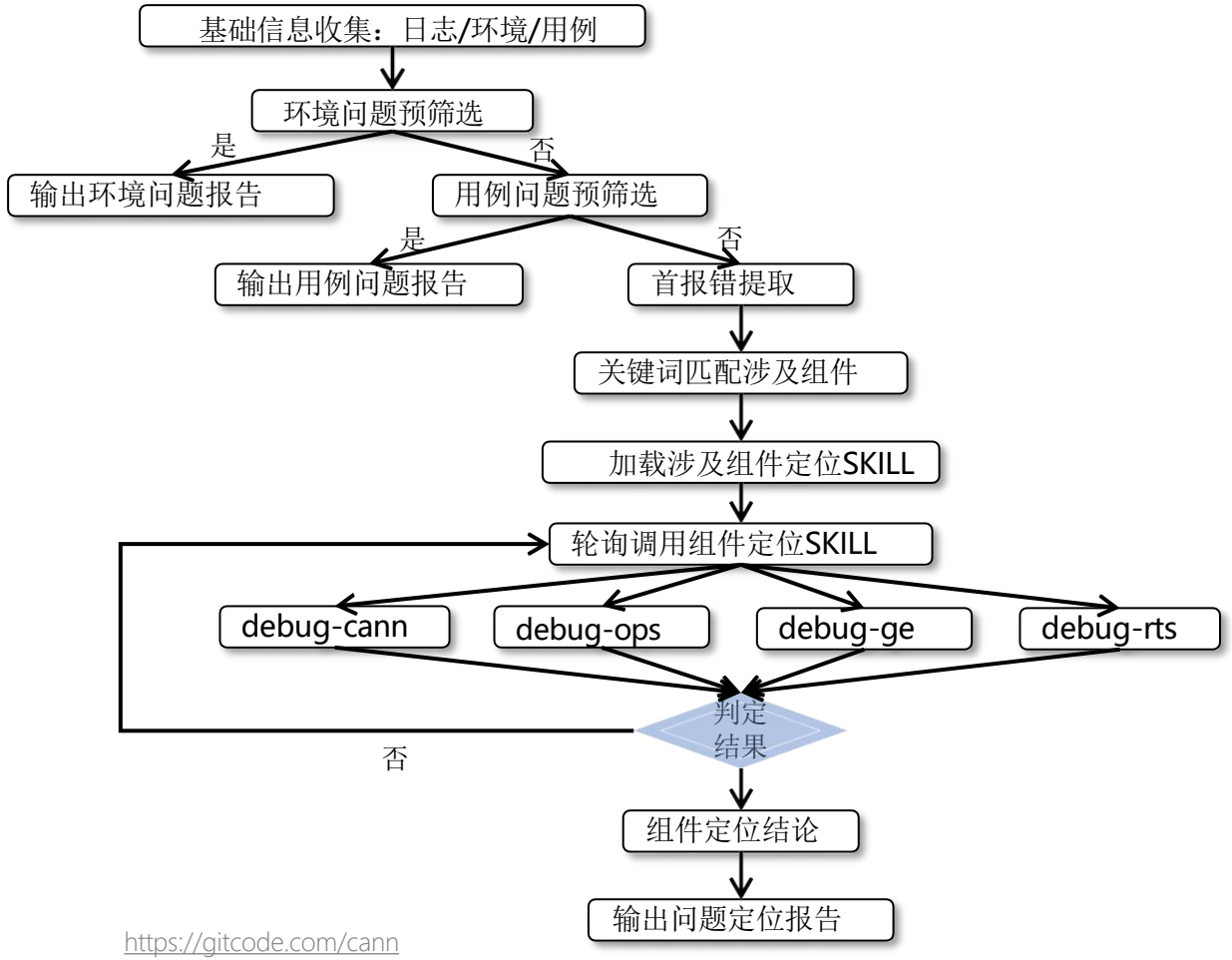
CANNBot算子测试：测试执行



CANNBot算子测试：问题定界定位

核心定位思路：

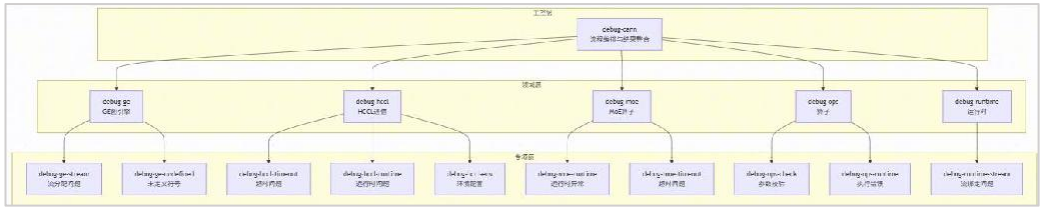
前置预筛选（环境问题/用例问题）→ 过滤首报错 → 匹配涉及组件 → 逐一判定组件归属 → Git Log变更追溯 → 深入定位具体问题类型 →三线交叉印证输出定位报告



https://gitcode.com/cann

说明：debug-cann / debug-ops / debug-ge / debug-rts 为组件维度的定位方式；调试可选用以下 Debug 技能：ascendc-crash-debug、ascendc-runtime-debug、ascendc-precision-debug、pypto-general-debug。

三层问题定位定界系统



流程关键节点说明：

节点	说明
收集Git版本信息	采集各组件代码仓的分支、commit hash、tag等版本标识，为后续变更追溯奠定基础
环境问题预筛选	快速检测硬件/驱动/CANN版本/网络/资源等环境异常
用例问题预筛选	快速检测API用法/参数/数据/逻辑等用例层面错误
提取首报错	从日志中定位时间最早、严重级别最高的错误信息
关键词匹配	根据首报错中的关键字匹配可能涉及的组件领域
组件技能判定	每个组件技能独立判断问题是否属于其专业领域
Git Log变更追溯	对疑似组件执行git log分析，追溯可疑commit变更，构建变更证据链
日志深度分析+源码追踪	日志分析与Git变更交叉印证，确认报错点与代码变更的因果关系
结果整合	汇总所有组件技能结论，按置信度和证据强度确定最终根因
输出报告	含根因组件、置信度、证据链（含Git commit证据）和修复建议

CANNBot算子测试：SKILL 工程看护

确保每个Skill在全生命周期内（上库前/上库后）的质量保障，通过分层策略和多维看护实现持续集成验证，覆盖正向/负向/正确性/流程/资源等五方面

▶ Skill 看护流程架构图

◆看护维度 (5 Dimensions of Skill Testing)

正向看护	负向看护	正确性看护	调用流程看护	资源消耗看护
显式/隐式均可 调用到该Skill	不该调用的提示词 不会误调用	黑盒场景验证 确保结果正确	关键工具被调用 交付件完整输出	Token消耗监控 防止资源浪费

◆分层看护策略 (Layered Testing Strategy)

UT（5分钟）	ST（1小时）	冒烟（3小时）
skill的规范性检查	skill端到端主流程守护	Cann-bench+评估体系

◆总体看护流程 (End-to-End Flow)

文档和脚本代码检查 Code Check → 前冒烟看护 Pre-commit Smoke → skill上库 Commit to Master → Daily看护 Post-commit Daily

修订方案：ascendc-st-design evals.json（共 12 个用例）

类别	子类	用例数	ID 范围
Cat-1: 触发边界验证	精准匹配	1	1
Cat-1: 触发边界验证	模糊匹配	1	2
Cat-1: 触发边界验证	噪音上下文匹配	1	3
Cat-1: 触发边界验证	负向匹配	1	4
Cat-2: 执行体验证	确定性步骤静态验证	4	5-8
Cat-2: 执行体验证	LLM 动态评分	4	9-12

Category 1: 触发边界验证（4 用例）

ID	name	prompt	expected_output	expectations
1	trigger_precise_match	"帮我设计 aclnnAdd 算子的ST测试用例，文档在 docs/aclnnAdd.md"	"执行参数清洗、因子提取、约束分析、L0/L1用例生成的完整5步流程，产出01-05中间文件及CSV"	contains: "01_parameter_description", contains: "04_constraints", contains: "L0", contains: "L1"
2	trigger_fuzzy_match	"这个算子要怎么测？帮我分析一下参数写出用例来，算子是aclnnAdd"	"识别为算子测试设计需求，执行参数分析→因子提取→约束编写→用例生成的完整流程"	contains: "参数", contains: "因子", contains: "用例", contains: "L0"
3	trigger_noisy_context	"你好啊，今天天气不错。对了领导刚安排了个活让我把aclnnAdd算子的测试用例搞一下，我之前没做过这种，文档已经放在 docs目录了。哦还有中午一起吃饭吗？"	"忽略闲聊天噪音，识别核心需求为aclnnAdd算子测试设计，执行完整测试设计流程"	contains: "01_parameter_description", not_contains: "吃饭", contains: "L0"
4	trigger_negative_match	"帮我生成 aclnnAdd 算子的 golden 函数，用于精度对比"	"识别为golden函数生成需求而非测试设计，提示该任任 不属于测试设计技能"	not_contains: "01_parameter_description", not_contains: "02_test_factors", not_contains: "04_constraints"

Category 2: 执行体验证（8 用例，同原方案 ID 13-20 调整为 5-12）

2.1 确定性步骤静态验证（ID 5-8）

ID	name	prompt	expectations
5	exec_static_step1_param_clean	"对 aclnnAdd 算子执行步骤1参数清洗"	contains: "01_parameter_description", contains: "参数名", contains: "数据类型", contains: "补充说明", not_contains: "### self"
6	exec_static_step2_factors	"根据 aclnnAdd 的参数说明生成测试因子 YAML"	contains: "aclnn_name", contains: "io_type", contains: "dtype", contains: "dimensions", contains: "exist"
7	exec_static_step3_constraints	"为 aclnnAdd 编写约束模块，包含场景枚举和 @solves 函数"	contains: "@solves", contains: "追溯", contains: "Candidates", contains: "from solver import", contains: "scenario"
8	exec_static_step4_generate	"对 aclnnAdd 执行约束校验和 L0/L1 用例生成"	contains: "generate_factor_values", contains: "--validate", contains: "generate_test_cases", contains: "L0", contains: "L1"

2.2 LLM 动态评分（ID 9-12）

ID	name	prompt	验证策略
9	exec_dynamic_full_pipeline	"帮我完成 aclnnAdd 算子的完整测试设计流程，从参数清洗到 L0/L1 用例生成"	use_new_session_validation: true, 评估5步流程完整性+每步质量
10	exec_dynamic_constraint_quality	"完成 aclnnSoftmax 算子的约束分析，场景枚举完整、追溯表覆盖所有约束、返回值符合类型契约"	use_new_session_validation: true, 评估约束实现质量
11	exec_dynamic_error_handling	"设计 aclnnFakeOp2026 算子的测试用例，文档在 docs/aclnnFakeOp2026.md"	use_new_session_validation: true + not_contains 防止虚构生成
12	exec_dynamic_partial_resume	"aclnnAdd 的 01 和 02 文件已经有了，从步骤3继续"	use_new_session_validation: true, 验证断点续做不重复生成

https://gitcode.com/cann

CANN

目录

Part 1 为什么算子Agent测试

Part 2 CANNBot算子测试全流程设计

Part 3 算子测试全流程实操

Part 4 开放性讨论

算子测试实操：示例



说明：演示相关截图只是示例，仓库会不停更新，具体以实际操作输出的为准

CANN

测试设计：参数清洗

```
# aclnnAbs 参数说明

## 基本信息

- **算子名称**：aclnnAbs
- **目标产品系列**：Ascend 950PR/Ascend 950DT
- **功能说明**：为输入张量的每一个元素取绝对值。计算公式：out = |self|

## 参数列表

| 参数名 | 类型 | 输入/输出 | 必选/可选 | 维度/长度 | 数据格式 | 数据类型 | 取值范围 | 空tensor | 非连续 | 补充说明 |
|--------|------|-----------|-----------|-----------|----------|----------|----------|----------|--------|----------|
| self | aclTensor | 输入 | 必选 | 0-8维 | ND | FLOAT、FLOAT16、DOUBLE、BFLOAT16、INT8、INT16、INT32、INT64、UINT8、BOOL、COMPLEX32、COMPLEX64 | 依赖具体数据类型 | 支持 | 支持 | 数据类型与out的dtype满足对应规则（见约束关系R2、R3）；产品特定限制：Atlas 训练系列产品、Atlas 推理系列产品不支持BFLOAT16、COMPLEX32、COMPLEX64（推断自：文档约束说明章节） |
| out | aclTensor | 输出 | 必选 | 0-8维 | ND | FLOAT、FLOAT16、DOUBLE、BFLOAT16、INT8、INT16、INT32、INT64、UINT8、BOOL | 不适用（输出参数） | 支持 | 支持 | shape与self相同（推断自：文档参数说明表"shape与self相同"）；数据类型与self的dtype满足对应规则（见约束关系R2、R3）；产品特定限制同self |
```

```
## 约束提取反向校验表

| R{n} | 约束项 | 原文片段 | 符号化 | 转录结果 | 状态 |
|------|--------|----------|--------|----------|------|
| R1 | out.shape与self相同 | "shape与self相同" | 一致 | shape与self相同 | EXACT_MATCH |
| R2 | 实数时dtype一致 | "当self为实数时，self和out的数据类型不一致" | 一致 | 实数时dtype相同 | EXACT_MATCH |
| R3 | 复数映射 | "COMPLEX64时，out的数据类型不为FLOAT32" | 映射 | COMPLEX64→float32, COMPLEX32→float16 | EXACT_MATCH |
| R4 | 维度范围 | "0-8" | ≤8 | 0-8维 | EXACT_MATCH |
| R5 | 产品限制 | "不支持BFLOAT16、COMPLEX32、COMPLEX64" | 排除 | 950产品全部支持 | EXACT_MATCH |
```


CANN

测试设计：测试因子提取

```yaml
operator_name: aclnnAbs

self:
  type: aclTensor
  io_type: input
  factors:
    self.exist: [true]
    self.format: [ND]
    # 来源: 01_parameter_description.md "数据格式"列 "ND"
    self.dimensions: [0, 1, 2, 3, 4, 5, 6, 7, 8]
    # 来源: 01_parameter_description.md "维度/长度"列 "0-8维"
    self.dtype: [float, float16, double, bfloat16, int8, int16, int32, int64, uint8, bool, complex32, complex64]
    # 来源: 01_parameter_description.md "数据类型"列

out:
  type: aclTensor
  io_type: output
  factors:
    out.exist: [true]
    out.format: [ND]
    # 来源: 01_parameter_description.md "数据格式"列 "ND"
    out.dimensions: [0, 1, 2, 3, 4, 5, 6, 7, 8]
    # 来源: 01_parameter_description.md "维度/长度"列 "0-8维"
    out.dtype: [float, float16, double, bfloat16, int8, int16, int32, int64, uint8, bool]
    # 来源: 01_parameter_description.md "数据类型"列 "FLOAT、FLOAT16、DOUBLE、BFLOAT16、INT8、INT16、INT32、INT64、UINT8、BOOL"
```


CANN

测试设计：场景枚举+约束实现

```
# aclnnAbs 场景枚举

## 约束关系清单

| ID | 约束描述 | 涉及参数 | 关键词 | 需要枚举 | @solves target |
|----|---------|---------|--------|---------|---------------|
| C1 | self和out的shape一致 | self.shape, out.shape | 一致 | 否 | out.shape |
| C2 | self为实数时dtype一致 | self.dtype, out.dtype | 一致(条件化) | 否 | out.dtype |
| C3 | 复数映射规则 | self.dtype, out.dtype | 映射表 | 是 | out.dtype |
| C4 | 维度范围 | self.dimensions | 范围 | 否 | – (factor-domain) |
| C5 | 产品dtype限制 | self.dtype, out.dtype | 排除 | 否 | – (factor-domain) |

## 子场景枚举

### C3: 复数映射规则

| 子场景 ID | 描述 | @solves 应生成的值示例 |
|----------|------|----------------------|
| C3-S1 | self.dtype为COMPLEX64 → out.dtype=float32 | self.dtype=complex64, out.dtype=float32 |
| C3-S2 | self.dtype为COMPLEX32 → out.dtype=float16 | self.dtype=complex32, out.dtype=float16 |
| C3-S3 | self.dtype为实数类型 → out.dtype与self.dtype一致 | self.dtype=float32, out.dtype=float32 |

## 空tensor维度场景

### 空tensor子场景
| 子场景 ID | 设置 | 输入shape | 输出shape | 输出状态 | 测试重点 |
|----------|------|----------|----------|---------|---------|
| ET-S1 | self首维=0 | self=[0,3] | out=[0,3] | 空 | 空→空 |

## 边界值场景

### Shape边界
| 子场景 ID | 参数 | 约束来源 | 边界值 | 维度 | 说明 |
|----------|------|---------|-------|------|------|
| BD-S1 | self | 默认兜底 | [1] | 0维 | 最小维度下边界 |
| BD-S2 | self | 默认兜底 | [1,1,1,1,1,1,1,1] | 8维 | 最大维度上边界 |
| BD-S3 | self | 默认兜底 | [1,2147483648] | 2维 | shape product上边界 |

### 值域边界
无显式值域约束参数，dtype边界由DEFAULT_VALUE_RANGES覆盖。

## 异常场景

### Shape/Array/枚举值/约束异常
| 子场景 ID | 异常类型 | 对应约束 | 违反方式 | 涉及参数 | 违反值 | _expected_error | 说明 |
|----------|---------|---------|---------|---------|-------|----------------|------|
| EX-S1 | shape_boundary | R4维度范围 | 取9维 | self.dimensions=9 | – | ACL_ERROR_INVALID_DIMENSION | 维度超出 |
| EX-S2 | constraint | R1 shape一致 | shape不匹配 | self.shape=[2,3], out.shape=[2,4] | – | ACL_ERROR_INVALID_SHAPE | shape不一致 |
| EX-S3 | constraint | R2 实数dtype一致 | dtype不匹配(实数) | self.dtype=float32, out.dtype=float16 | – | ACL_ERROR_DTYPES_NOT_MATCH | 实数dtype不一致 |
| EX-S4 | constraint | R3 复数映射 | 复数映射不匹配 | self.dtype=complex64, out.dtype=float16 | – | ACL_ERROR_DTYPES_NOT_MATCH | 复数映射错误 |
| EX-S5 | enum_range | 不支持dtype | 无效dtype | self.dtype=int4 | – | ACL_ERROR_INVALID_PARAM | 不支持的dtype |
```


CANN

测试设计：约束校验+用例生成

```python
# ===== 追溯表结束 =====

# Complex -> real dtype mapping table (source: 01_parameter_description.md R3)
_COMPLEX_TO_REAL_MAP = {
    'complex64': 'float',       # COMPLEX64 -> FLOAT32
    'complex32': 'float16',     # COMPLEX32 -> FLOAT16
}

_REAL_DTYPES = ['float', 'float16', 'double', 'bfloat16', 'int8', 'int16', 'int32', 'int64', 'uint8', 'bool']
_COMPLEX_DTYPES = ['complex32', 'complex64']


@solves('out.shape', sources=['self.shape'])
def solve_out_shape(self_shape):
    """R1: out.shape equals self.shape"""
    if self_shape is None:
        return NOT_APPLICABLE
    assert len(self_shape) <= 8, f"self dimensions {len(self_shape)} exceeds 8"
    return list(self_shape)


@solves('out.dtype', sources=['self.dtype'])
def solve_out_dtype(self_dtype):
    """R2+R3: Real dtype equal; Complex dtype maps to corresponding real dtype"""
    if self_dtype is None:
        return NOT_APPLICABLE

    if self_dtype in _COMPLEX_TO_REAL_MAP:
        # R3: Complex mapping rule
        mapped = _COMPLEX_TO_REAL_MAP[self_dtype]
        tag('scenario', 'C3-S1' if self_dtype == 'complex64' else 'C3-S2')
        return mapped
    else:
        # R2: Real type - out.dtype equals self.dtype
        tag('scenario', 'C3-S3')
        assert self_dtype in _REAL_DTYPES, f"Unknown real dtype: {self_dtype}"
        return self_dtype


@solves('out.dimensions', sources=['self.dimensions'])
def solve_out_dimensions(self_dimensions):
    """out.dimensions equals self.dimensions (same tensor shape constraint)"""
    if self_dimensions is None:
        return NOT_APPLICABLE
    return self_dimensions
```

```python
def validate_constraints(case: dict) -> list[str]:
    """Validate all R{n} constraints for a given test case.
    Returns list of violated constraint IDs.

    R1: self and out shapes must be identical
    R2: when self is real type, self.dtype == out.dtype
    R3: complex type mapping (COMPLEX64->float, COMPLEX32->float16)
    R4: self dimensions <= 8
    R5: product dtype restriction (950PR/950DT all supported, pass-through)
    """
    violations = []

    # R1: shape一致
    self_shape = case.get('self.shape')
    out_shape = case.get('out.shape')
    if self_shape is not None and out_shape is not None:
        if list(self_shape) != list(out_shape):
            violations.append('R1')

    # R2: 实数时dtype一致
    self_dtype = case.get('self.dtype')
    out_dtype = case.get('out.dtype')
    if self_dtype is not None and out_dtype is not None:
        if self_dtype in _REAL_DTYPES:
            if self_dtype != out_dtype:
                violations.append('R2')

    # R3: 复数映射
    if self_dtype is not None and out_dtype is not None:
        if self_dtype in _COMPLEX_TO_REAL_MAP:
            expected = _COMPLEX_TO_REAL_MAP[self_dtype]
            if out_dtype != expected:
                violations.append('R3')
```


CANN

测试编码

现在我已经完成了所有信息的收集和分析。让我整理算子分析结果：

算子分析

项目	分析结果
算子功能	为输入张量每个元素取绝对值：`out =
参数	self(aclTensor*, 输入) + out(aclTensor*, 输出)
参数类型	全部为 Tensor，无 Scalar
对标实现	使用 torch.abs() PyTorch 原生接口（单 Tensor 运算）
是否需要 Input Func	❯ 不需要（非反向算子，无参数转换）
是否升精度	❯ 不升精度（文档未提及）
置信度	★★★★★（完全等价的 PyTorch 原生 API）
异常预防	无需（abs 无除零/溢出风险）

现在创建输出目录并生成 Golden 文件：

CANN

测试编码

验证报告：

[语法检查] ✓ PASS
[PyTorch API] ✓ PASS - 未使用 .dtype.name/.astype()/.numpy()
[注册字典] ✓ PASS - __golden__ 格式正确: {"aclnn": {"aclnnAbs": "aclnn_abs_golden"}}
[参数签名] ✓ PASS - self(camelCase), out=None(可选参数有默认值), **kwargs
[实现/精度] ✓ PASS - 文档未提及升精度，直接使用 torch.abs() 计算
[异常预防] ✓ PASS - abs 无除零/溢出风险，无需 epsilon 保护
[置信度评估] ✓ PASS → ⋆f⋆c⋆m⋆l⋆a (使用 PyTorch 原生接口 torch.abs，完全等价)
[Input Func 判断] ✓ PASS → 不需要，原因：非反向算子，无参数转换需求

📦 输出路径

文件	路径	状态
Golden 文件	./inputs/aclnnAbs/golden/ttk/aclnnAbs.py	✓ PASS
Input Func 文件	✗ SKIP → 不需要	

🎯 Golden 实现核心逻辑说明

1. 核心计算公式：out = |self|，直接调用 torch.abs(self) 实现
2. PyTorch 原生接口对标：torch.abs() 与 aclnnAbs 完全等价
   - 对实数类型（float16/float32/float64/bfloat16/int8/int16/int32/int64/uint8/bool）→ 返回同 dtype 的绝对值
   - 对复数类型（complex64）→ 返回 float32 的模（magnitude）
   - 对复数类型（complex32/chalf）→ 返回 float16 的模
3. 无需升精度：文档未提及精度提升要求，直接使用原始 dtype 计算
4. 无需异常预防：abs 运算不存在除零、溢出等风险
5. 完成判定：./inputs/aclnnAbs/golden/ttk/aclnnAbs.py 文件已存在 ✓

CANN

覆盖度评估

覆盖度评估结果汇总

维度	覆盖度	评级
参数组合覆盖度	100.0%	✓ 达标
dtype覆盖度	100.0%	✓ 达标
format覆盖度	100.0%	✓ 达标
shape覆盖度（字节区间）	92.9%	✓ 良好
shape覆盖度（维度）	100.0%	✓ 达标
数据范围覆盖度	28.0%	✗ 不足
attr值覆盖度	N/A	-

测试输出

aclnnAbs 算子测试全流程已完成！ 以下是各阶段输出路径汇总：

阶段	输出路径
配置文件	./config.json
测试用例	./inputs/aclnnAbs/tests/st/testcases/aclnnAbs_l0_functional.csv (40条)
	./inputs/aclnnAbs/tests/st/testcases/aclnnAbs_l1_functional.csv (514条)
	./inputs/aclnnAbs/tests/st/testcases/aclnnAbs_l1_rdfunctional.csv (1000条)
Golden脚本	./inputs/aclnnAbs/golden/ttk/aclnnAbs.py
执行结果	./outputs/aclnnAbs/aclnnAbs_result_20260611_122820.csv (35通过/5失败)
覆盖度报告	./outputs/aclnnAbs/aclnnAbs_coverage_evaluation_20260611_122820.md
问题分析报告	./outputs/aclnnAbs/aclnnAbs_result_analysis_20260611_122820.md
测试报告	./outputs/aclnnAbs/aclnnAbs_report_20260611_122820.md

关键结论：40条L0用例，87.5%通过率。5个失败用例：2个bool类型Golden问题（用例问题），3个complex32 NPU执行问题（版本问题）。

CANN

目录

Part 1 为什么算子Agent测试

Part 2 CANNBot算子测试全流程设计

Part 3 算子测试全流程实操

Part 4 开放性讨论

开放性讨论

- 探索测试：当前算子Agent测试还是遵循固定的workflow和SKILL展开，Agent如何走向自主探索测试。
- 测试完备度：如何评估Agent测试的完备度，除了覆盖度评估外是否还有其他量化指标。
- 黑白盒验证：Agent如何开展黑白盒测试，函数/分支级的黑白盒验证是否还有意义。
- Agent测试：当前针对SKILL做了门禁机制，如何对端到端对Agent系统做看护。

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

CANN Be X计划之赏金猎人【夏季】招募开启 任务一：【新手村礼遇】

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann